GRPO Baseline Training Experiment Report

Establishing a Reinforcement Learning Baseline for

Adaptive KL Regularization via Meta-Learning

Prepared by: Ruth Tamiru

Submitted to: Dr. Beakal Gizachew

Date: March 19, 2026

Abstract

This report documents the design, implementation, and results of a baseline Group Relative Policy Optimization (GRPO) training experiment conducted as part of my MSc thesis on Adaptive KL Regularization via Meta-Learning. The primary objective was to establish a reproducible performance baseline on the MATH500 benchmark using the Qwen2.5-0.5B language model trained within the `verl` framework on a single NVIDIA RTX A4000 GPU. The experiment involved overcoming significant GPU memory constraints through systematic hyperparameter tuning, and ultimately produced a final validation accuracy of **29.5%** after one full training epoch, compared to a pre-training baseline of 25.3%. This result provides the empirical foundation against which the proposed MetaKLController — an LSTM-based adaptive KL coefficient mechanism — will be evaluated in subsequent experiments.

Contents

1. Introduction

Reinforcement learning from human feedback (RLHF) and its derivative methods have become central to aligning large language models (LLMs) with desired behaviors [1]. Among these, Group Relative Policy Optimization (GRPO) has emerged as a particularly effective approach for improving mathematical reasoning capabilities in LLMs, as demonstrated by its use in training the DeepSeek-R1 family of models [2]. Unlike Proximal Policy Optimization (PPO) [3], which requires a separate critic model, GRPO estimates advantages by comparing a group of sampled responses relative to each other, making it computationally more efficient for single-GPU settings [4].

A core challenge in GRPO — and in policy gradient methods more broadly — is managing the KL divergence penalty between the current policy and a reference policy. This penalty term prevents the model from drifting too far from its pre-trained behavior, but its optimal coefficient is highly dependent on the training stage, task difficulty, and model capacity [5]. Current implementations typically use a fixed KL coefficient throughout training, which is a strong and potentially suboptimal assumption.

The central contribution of my thesis is the **MetaKLController**: a meta-learning approach that uses a Long Short-Term Memory (LSTM) network [6] to dynamically predict the optimal KL coefficient at each training step based on observed training signals. Before this adaptive mechanism can be evaluated, a stable and reproducible baseline must first be established using the standard fixed-coefficient GRPO approach. This report documents that baseline experiment in full detail.

2. Background and Related Work

2.1 Group Relative Policy Optimization (GRPO)

GRPO was introduced as a memory-efficient alternative to PPO for LLM fine-tuning [2]. Given a prompt, the policy generates a group of G responses. The advantage for each response is computed as:

$$A_i = \frac{r_i - \mu_r}{\sigma_r} \tag{1}$$

where r_i is the reward of the i-th response, μ_r is the group mean reward, and σ_r is the group standard deviation. The full GRPO objective is:

$$\mathcal{L}(\theta) = \mathbb{E}\left[A_i \cdot \log \pi_\theta(o_i \mid q)\right] - \beta \cdot \mathrm{KL}\left(\pi_\theta \,\|\, \pi_{\text{ref}}\right) \tag{2}$$

where β is the KL coefficient, π_θ is the current policy, and π_{ref} is the frozen reference policy. In standard implementations, β is a fixed scalar hyperparameter set before training begins [2, 4].

2.2 Mathematical Reasoning Benchmarks

The MATH benchmark [7] and its 500-problem evaluation subset MATH500 have become standard evaluation sets for measuring mathematical reasoning in LLMs. Problems span seven difficulty levels across topics including algebra, geometry, number theory, and calculus. The benchmark is particularly useful for GRPO training because answers can be automatically verified using exact symbolic matching, enabling fully automated reward computation without human annotation.

2.3 SimpleRL-Zoo Framework

This experiment builds upon the SimpleRL-Zoo codebase [4], which provides clean, minimal reimplementations of GRPO and related RL training recipes for LLMs. SimpleRL-Zoo reports that training Qwen2.5-0.5B with standard GRPO improves MATH500 accuracy from approximately 15.8% to 34.4% over multiple epochs. Our experiment uses the same model and dataset but adapts the configuration for a single-GPU resource-constrained environment.

2.4 The `verl` Framework

The Vector Environment for Reinforcement Learning (`verl`) framework [8] provides the distributed training infrastructure used in this experiment. `verl` implements a hybrid architecture combining Fully Sharded Data Parallelism (FSDP) for the training model and the `vllm` inference engine [9] for efficient rollout generation. This hybrid design introduces a specific constraint on single-GPU deployments: both the training copy and inference

copy of the model must simultaneously reside on GPU during weight synchronization, effectively doubling the minimum GPU memory requirement.

3. Experimental Setup

3.1 Hardware and Software Environment

All experiments were conducted on a single workstation. Table 1 summarizes the hardware and software environment.

Table 1: Hardware and software environment

Component	**Specification**
GPU	NVIDIA RTX A4000, 16 GB VRAM
CPU	52-core processor
System RAM	64 GB
Operating System	Ubuntu 24
Python	3.10 (Conda environment: `ruth_verl`)
Deep Learning Framework	PyTorch with CUDA 12.4
Training Framework	`verl` (FSDP + vllm hybrid)
Inference Engine	vllm 0.6.3
NCCL Version	2.20.5

3.2 Model

The base model used in this experiment is Qwen2.5-0.5B [10], a 494 million parameter causal language model. The model was selected because its parameter count (˜1 GB in bfloat16) fits within the memory budget of a single RTX A4000 under the dual-copy constraint of the `verl` hybrid architecture, and it serves as a documented baseline in the SimpleRL-Zoo codebase [4].

Table 2: Qwen2.5-0.5B model architecture

Parameter	**Value**
Architecture	Qwen2ForCausalLM
Total parameters	494 million
Hidden size	896
Number of layers	24
Attention heads	14
Key-value heads	2 (Grouped Query Attention)
Vocabulary size	151,936
Maximum context length	32,768 tokens
Training precision	bfloat16

3.3 Dataset

The training data consists of mathematical problem-answer pairs from the MATH dataset [7], preprocessed into the parquet format required by `verl`. After applying the prompt format filter, the dataset statistics are as follows:

Table 3: Dataset statistics after filtering

Split	**Original**	**Filtered**	**Filter Criterion**
Training	8,523	7,119	Prompt length $\leq$ 512 tokens
Validation	500	434	MATH500 subset

With a training batch size of 16, this yields **444 training steps per epoch**. Each step processes 16 prompts, generates 2 responses per prompt (n=2), computes rewards using exact answer matching, and performs one gradient update.

3.4 GPU Memory Analysis and Model Selection

A significant portion of the experimental work involved diagnosing and resolving GPU out-of-memory (OOM) errors. The `verl` framework loads the model in two forms simultaneously: an FSDP-wrapped version for gradient computation and a vllm-managed version for inference. During the weight synchronization step (`sync_model_weights`), both copies must reside on GPU concurrently, effectively doubling the minimum VRAM requirement.

For Qwen2.5-1.5B, the estimated peak memory during synchronization exceeds 15 GB, surpassing the 15.6 GB capacity of the RTX A4000. Table 4 summarizes the systematic experiments conducted to identify a stable configuration.

Table 4: OOM diagnosis experiments and outcomes

Configuration	**Batch**	**n**	**Steps OK**	**Failure Point**
1.5B, default settings	32	4	0	Weight sync initialization
0.5B, n=4, batch=32	32	4	1	Rollout `repeat_interleave`
0.5B, n=2, batch=32	32	2	1	Rollout `repeat_interleave`
0.5B, n=2, batch=16	16	2	10	Log-prob rank computation
0.5B, n=2, batch=16, `gpu_util=0.4`	16	2	444 ✓	Completed

The critical fix was reducing `gpu_memory_utilization` from 0.5 to 0.4, which instructs `vllm` to reserve less GPU memory for its KV cache, leaving sufficient headroom for FSDP training operations between rollout batches.

3.5 Final Training Configuration

Table 5: Final hyperparameter configuration

Hyperparameter	**Value**
Algorithm	GRPO
Base model	Qwen2.5-0.5B
Training batch size	16
Responses per prompt (n)	2
Maximum prompt length	512 tokens
Maximum response length	512 tokens
Learning rate	5×10^{-7}
KL loss coefficient (β)	1×10^{-4}
KL loss type	`low_var_kl`
Entropy coefficient	1×10^{-3}
PPO clip ratio	0.2
vllm GPU memory utilization	0.4
Actor parameter offload	Enabled
Reference model parameter offload	Enabled
Checkpoint save frequency	Every 10 steps
Resume mode	Auto (from latest checkpoint)
Total epochs	1
Total training steps	444

4. Methodology

4.1 Training Pipeline

Each training step follows the standard GRPO pipeline as implemented in `verl`:

1. **Rollout generation:** For each batch of 16 prompts, the `vllm` inference engine generates 2 responses per prompt (32 total), using temperature = 1.0 for stochastic sampling.
2. **Reward computation:** Each response is scored using a mixed reward function that awards +1.0 for a correctly formatted and correct answer, and applies a format penalty of −1.0 for responses that do not contain a properly formatted `\boxed{}` expression.
3. **Advantage estimation:** GRPO advantages are computed within each group of 2 responses. The advantage of each response is normalized relative to the group mean and standard deviation as in Equation (1).
4. **Reference log-probabilities:** The frozen reference model computes token log-probabilities for all generated responses, enabling the KL divergence term to be calculated during the actor update.
5. **Actor update:** The policy parameters are updated using the clipped surrogate objective with clip ratio 0.2, plus the KL loss weighted by $\beta = 10^{-4}$, plus entropy regularization weighted by 10^{-3}.

4.2 Evaluation Protocol

Evaluation is performed at step 0 (pre-training baseline) and at step 444 (end of epoch). The metric `val/test_score` is defined as the fraction of MATH500 problems for which the model produces a correctly boxed answer matching the ground truth. Evaluation uses greedy decoding over the 434 filtered validation problems.

4.3 Reward Function

The reward function combines two components:

- **Correctness reward:** +1.0 if the model's response contains a `\boxed{}` expression

that exactly matches the ground truth after symbolic normalization.

- **Format penalty:** -1.0 if the response does not contain a properly formatted `\boxed{}` expression, regardless of correctness.

5. Results

5.1 Pre-Training Baseline

Before any gradient updates, Qwen2.5-0.5B was evaluated on MATH500 to establish a pre-training baseline. Two evaluation runs yielded scores of 25.1% and 23.7%, giving an average of approximately **25.3%**. This variation is expected due to stochastic response generation and dataset shuffling.

The baseline is higher than the 15.8% reported by SimpleRL-Zoo for the same model, likely reflecting differences in prompt template and answer extraction logic. The absolute baseline value is less important than the improvement measured from it.

5.2 Post-Training Result

After completing the full training epoch of 444 steps, the model achieved a final validation score of **29.5%** on MATH500. Table 6 summarizes the key results.

Table 6: Summary of experimental results

Metric	**Value**
Pre-training baseline (step 0)	25.3%
Post-training score (step 444)	**29.5%**
Absolute improvement	+4.2 percentage points
Relative improvement	+16.6%
Training steps completed	444 / 444 (100%)
Average step time	≈33 seconds
Total training time	≈4 hours 5 minutes

5.3 Training Dynamics

The training remained numerically stable throughout the full epoch. Key observations include:

- The actor KL loss remained consistently near 0.001 throughout training, indicating that the policy did not drift excessively from the reference model with $\beta = 10^{-4}$.
- The actor gradient norm was generally below 2.0, suggesting stable gradient updates.
- The average reward per step (`critic/score/mean`) showed gradual improvement from ≈0.062 in early steps, consistent with the model learning to produce correctly formatted and correct answers more frequently.
- The response clip ratio (fraction of responses truncated at 512 tokens) remained between 37.5% and 75%, indicating that many responses were cut off before completion.

5.4 Comparison with Reference Results

Table 7: Comparison with SimpleRL-Zoo reference results

Configuration	**Baseline**	**Post-Training**	**Improvement**
SimpleRL-Zoo (reported) [4]	15.8%	34.4%	+18.6%
This experiment (1 epoch)	25.3%	29.5%	+4.2%

The smaller absolute improvement in our experiment is attributable to two factors: (1) the higher starting baseline leaves less room for improvement within a single epoch, and (2) the SimpleRL-Zoo results likely reflect multiple training epochs. The relative improvement of +16.6% is consistent with expected single-epoch GRPO dynamics.

6. Discussion

6.1 Significance of the Baseline

The successful completion of this baseline experiment serves three important purposes for the thesis. First, it validates that the complete GRPO training pipeline is functioning correctly in our experimental environment. Second, it provides a concrete numerical target (29.5% after 1 epoch with fixed $\beta = 10^{-4}$) against which the proposed MetaKLController will be benchmarked. Third, it establishes the infrastructure — including checkpointing, auto-resume, and autonomous restart on server reboot — necessary for the longer experiments that will follow.

6.2 The Fixed KL Coefficient Problem

One of the most instructive observations from this experiment concerns the fixed KL coefficient. While $\beta = 10^{-4}$ produced stable training, it was chosen manually through intuition rather than principled optimization. The training logs reveal that the KL loss remained consistently near 0.001 throughout all 444 steps, suggesting that the regularization strength was identical during early training — when the policy is far from optimal and might benefit from less constraint — and during later training, when the policy is closer to optimal and might benefit from stronger regularization to prevent reward hacking.

This observation directly motivates the MetaKLController. An LSTM-based controller that observes current KL loss, reward trends, and gradient statistics could learn to increase β when reward improvement stagnates (indicating potential reward hacking) and decrease β when the model is still making rapid genuine progress. The baseline result of 29.5% with fixed $\beta = 10^{-4}$ thus becomes the comparison point for testing whether adaptive β scheduling meaningfully improves upon this.

6.3 Limitations

- **Single epoch:** Full convergence likely requires 3–5 epochs. Multi-epoch results will be reported in the full thesis.
- **Response length truncation:** The high clip ratio (37.5–75%) suggests many responses were cut off at 512 tokens, potentially suppressing performance.
- **Single model size:** Only Qwen2.5-0.5B was evaluated due to hardware constraints. The 1.5B model would provide a more robust baseline.
- **No hyperparameter search:** The KL coefficient $\beta = 10^{-4}$ was used without

systematic tuning, which limits the strength of the fixed-coefficient baseline.

7. Next Steps

The completion of this baseline experiment enables the following next steps:

1. **Implement MetaKLController:** Design and implement the LSTM-based adaptive KL coefficient predictor. The controller will take as input a window of recent training statistics (KL loss, reward mean, reward std, gradient norm) and output an adapted β value for the next training step.
2. **Integration with `verl`:** Modify `verl/workers/actor/dp_actor.py` to replace the static `self.config.kl_loss_coef` with a call to the MetaKLController at each training step.
3. **Comparative experiments:** Train Qwen2.5-0.5B with the MetaKLController under identical conditions and measure the difference in final MATH500 accuracy and convergence speed.
4. **Multi-epoch baseline:** Run the standard GRPO baseline for 3 epochs to establish a stronger fixed-coefficient ceiling for comparison.
5. **Ablation study:** Evaluate the sensitivity of the MetaKLController to its own hyperparameters (LSTM hidden size, input window length, output range).

8. Conclusion

This report has documented the complete methodology and results of a GRPO baseline training experiment conducted on the MATH500 benchmark using Qwen2.5-0.5B. Despite significant GPU memory constraints on a single RTX A4000, a stable training configuration was identified through systematic experimentation, and a full training epoch of 444 steps was successfully completed. The model improved from a pre-training accuracy of 25.3% to a post-training accuracy of **29.5%**, representing a relative improvement of 16.6%.

References

[1] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). *Training language models to follow instructions with human feedback.* Advances in Neural Information Processing Systems, 35, 27730–27744.

[2] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., ... & He, Y. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv preprint arXiv:2501.12948.

[3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms.* arXiv preprint arXiv:1707.06347.

[4] SimpleRL-Zoo Contributors. (2025). *SimpleRL-Zoo: Scaling RL Training on Open-Source LLMs with Verifiable Rewards.* GitHub Repository: https://github.com/hkust-nlp/simpleRL-reason.

[5] Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., ... & Irving, G. (2019). *Fine-Tuning Language Models from Human Preferences.* arXiv preprint arXiv:1909.08593.

[6] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), 1735–1780.

[7] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., ... & Steinhardt, J. (2021). *Measuring Mathematical Problem Solving with the MATH Dataset.* Advances in Neural Information Processing Systems, 34, 12537–12551.

[8] Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., ... & Zaharia, M. (2024). *verl: A Flexible, Efficient and Production-Ready RL Training Framework for LLMs.* GitHub Repository: https://github.com/volcengine/verl.

[9] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 611–626.

[10] Qwen Team. (2024). *Qwen2.5: A Party of Foundation Models.* Qwen Blog, Alibaba Cloud. https://qwenlm.github.io/blog/qwen2.5/.